

Sistemi informativi su Web

Persistenza: da JDBC a CrudRepository aa 2024-2025



Paolo Merialdo
Università degli Studi Roma Tre



This work is licensed under a Creative Commons Attribution-NonCommercial-Share Alike 3.0 United States
See <http://creativecommons.org/licenses/by-nc-sa/3.0/us/> for details



JDBC

JDBC: Concetti Fondamentali

- Introduzione
 - Architettura
- Un Esempio
 - Driver e Driver Manager
 - Connection
 - PreparedStatement
 - ResultSet
 - SQLException
- Aspetti metodologici
 - Lazy load
 - Id Broker

Introduzione

- Da Java Tutorial
 - *The JDBC API is a Java API that can access any kind of tabular data, especially data stored in a Relational Database*
 - *JDBC helps you to write java applications that manage these three programming activities:*
 - *Connect to a data source, like a database*
 - *Send queries and update statements to the database*
 - *Retrieve and process the results received from the database in answer to your query*

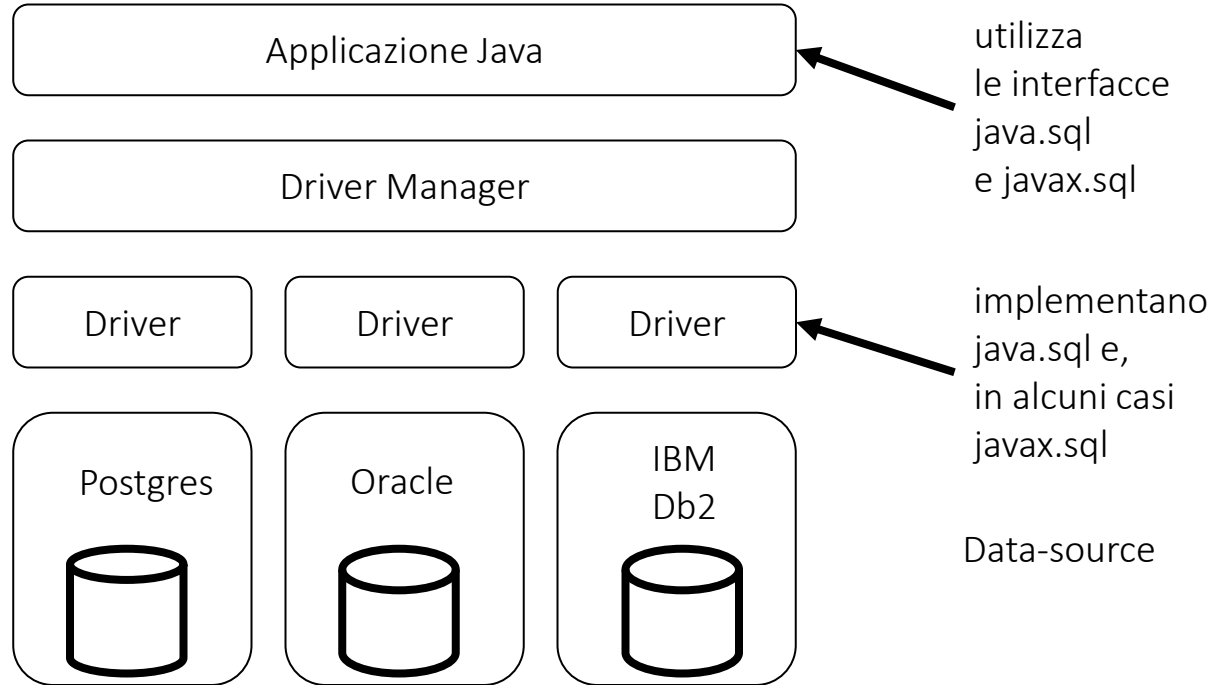
Introduzione

- Un'applicazione che accede (in lettura e/o scrittura) ad una sorgente di dati (nel nostro caso un database relazionale) ha bisogno di fare le seguenti operazioni:
 1. aprire una "**connessione**" alla sorgente dati
 2. inviare attraverso la connessione istruzioni (di interrogazione o di aggiornamento) alla sorgente dati
 3. processare i risultati ricevuti dalla sorgente dati in risposta alle istruzioni inviate

Introduzione

- Le nostre sorgenti dati sono database gestiti da un DBMS
- Ogni DBMS espone una interfaccia attraverso la quale può ricevere comandi SQL e restituire i risultati della loro esecuzione
- Le applicazioni interagiscono con questa interfaccia attraverso un **driver**
- Possiamo pensare il driver come un interprete che permette all'applicazione di dialogare con il DBMS attraverso un'interfaccia standard (JDBC)

Architettura



Esempio

- Obiettivo: esemplificare l'API JDBC
 - scriviamo codice che ha unicamente lo scopo di evidenziare l'uso della API JDBC
 - non ci preoccupiamo (troppo) della qualità del codice
- Operazioni
 - inserimento di una tupla nel db
 - cancellazione di una tupla
 - ricerca di un tupla per chiave primaria
 - ricerca di un insieme di tuple (per qualche proprietà)

Esempio

- Consideriamo la classe **Studente**:

```
package it.uniroma3.model;

import java.util. Date;
public class Studente {
    private String matricola;
    private String nome;
    private String cognome;
    private Date dataNascita;

    public Studente(){}
    public String getNome() {
        return this.nome;
    }
    public void setName(String nome) {
        this.nome = nome;
    }
    // seguono gli altri metodi getter e setter
}
```

Esempio

Per memorizzare i suoi dati creiamo una tabella in un database relazionale

```
CREATE DATABASE university;
```

```
CREATE TABLE studente
```

```
(
```

```
    matricola character(8),
```

```
    nome character varying(64) NOT NULL,
```

```
    cognome character varying(64) NOT NULL,
```

```
    datanascita date NOT NULL,
```

```
    CONSTRAINT pk_studente PRIMARY KEY (matricola)
```

```
)
```

Le classi fondamentali di JDBC

- Classe **DriverManager**
- Interfaccia **Driver**
- Interfaccia **Connection**
- Interfaccia **PreparedStatement**
- Interfaccia **ResultSet**
- Eccezione **SQLException**

Un Esempio

- Nel seguito, con riferimento ad nostro esempio, descriveremo le seguenti operazioni
 - Operazione n.1: Caricare il driver
 - Operazione n.2: Aprire una connessione
 - Operazione n.3: Definire l'istruzione SQL
 - Operazione n.4: Gestire il risultato
 - Operazione n.5: Gestire le eccezioni

Operazione n.1

Caricare il Driver

- Creare un oggetto della classe **Driver**
`Driver d = new org.postgresql.Driver();`
- Registrare il driver sul **DriverManager**
`void DriverManager.registerDriver(Driver d);`
- A questo punto il driver è disponibile

Operazione n.1

Caricare il Driver

- Le operazioni precedenti sono equivalenti alla istruzione:

```
Class.forName("org.postgresql.Driver");
```

Operazione n.2

Ottenere una Connessione

- Ottenere una connessione dal **DriverManager**
 - per farlo è necessario specificare:
 - host, dbms, db
 - utente e password
- URI (indirizzo) della connessione
 - specifica server, porta e database
- Sintassi
 - `jdbc:<sottoprotocollo>:<parametri>`**

Operazione n.2

Ottenere una Connessione

- URI per Postgres

```
jdbc:postgresql:<baseDati>
```

```
jdbc:postgresql://<host>/<baseDati>
```

```
jdbc:postgresql://<host>:<porta>/<baseDati>
```

- Esempi

```
jdbc:postgresql:university
```

```
jdbc:postgresql://127.0.0.1/university
```

```
jdbc:postgresql://193.204.161.14:5432/university
```

```
jdbc:odbc:university
```

Operazione n.2

Ottenere una Connessione

- Creazione della connessione

```
Connection DriverManager.getConnection(  
    String uri,  
    String utente,  
    String password);
```

- Esempio

```
Connection connection;  
connection = DriverManager.getConnection(  
    "jdbc:postgresql:university" ,  
    "postgres",  
    "postgres" );
```

- Attenzione: ottenere una connessione è un'operazione costosa
 - Creare troppe connessioni comporta problemi di prestazioni
 - Esistono tecniche (connection pooling) per superare questo problema (ma non le vedremo)

Nell'esempio

- Confiniamo nella classe **DataSource** la responsabilità di gestire le operazioni necessarie per ottenere la connessione
 - il suo compito è servire connessioni alle altre classi che ne hanno bisogno
 - metodo **Connection getConnection()** che restituisce una nuova connessione ad ogni richiesta
- E' una soluzione artigianale usata a fini didattici (ma ha una sua dignità)

La classe DataSource

```
package it.uniroma3.persistence;
import java.sql.*;

public class DataSource {
    private String dbURI = "jdbc:postgresql://localhost/university";
    private String userName = "postgres";
    private String password = "postgres";

    public Connection getConnection() throws PersistenceException {
        Connection connection = null;
        try {
            Class.forName("org.postgresql.Driver");
            connection = DriverManager.getConnection(dbURI, userName, password);
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return connection;
    }
}
```

Operazione n.3

Istruzione SQL

- Vediamo ora il codice JDBC che esegue istruzioni SQL per:
 - salvare (rendere persistenti) oggetti nel db
 - cancellare oggetti dal db
 - trovare oggetti dal db
- Confiniamo il codice di queste operazioni in una classe che ha la responsabilità di gestire l'accesso al database per gli oggetti Studente: la classe **StudenteDAO**
- DAO è acronimo di Data Access Object

Operazione n.3

Istruzione SQL

- Per eseguire una istruzione SQL è necessario creare un oggetto della classe che implementa **PreparedStatement**
 - creato dall'oggetto **Connection** invocando il metodo:
PreparedStatement preparedStatement(String s);
- La stringa s è una istruzione SQL parametrica: i parametri sono indicati con il simbolo ?
- Esempio 1

```
String insert = "insert into studente(nome, cognome,
datanascita, matricola) values (?, ?, ?, ?)";
PreparedStatement statement =
connection.prepareStatement(insert);
```
- Esempio 2

```
String delete = "delete from studente where
matricola=?";
PreparedStatement statement =
connection.prepareStatement(delete);
```

Operazione n.3

Istruzione SQL

- I parametri sono assegnati mediante opportuni metodi della classe che implementa **PreparedStatement**
 - metodi `setXXX (<numPar>, <valore>)`
 - un metodo per ogni tipo, il primo argomento corrisponde all'indice del parametro nella query, il secondo al valore da assegnare al parametro

- Esempio 1 (cont.)

```
PreparedStatement statement;  
String insert = "insert into studente(nome, cognome, datanascita,  
matricola) values (?, ?, ?, ?)";
```

```
statement = connection.prepareStatement(insert);  
statement.setString(1, studente.getNome());  
statement.setString(2, studente.getCognome());
```

```
long secs = studente.getDataNascita().getTime();  
statement.setDate(3, new java.sql.Date(secs));
```

```
statement.setString(4, studente.getMatricola());
```

Operazione n.3

Istruzione SQL

- JDBC usa **java.sql.Date**, mentre la classe Studente usa **java.util.Date**

- Le istruzioni

```
long secs = student.getBirthDate().getTime();  
statement.setDate(4, new java.sql.Date(secs));
```

servono a "convertire" una data da una rappresentazione all'altra

- Per i dettagli vedi la documentazione delle librerie

Operazione n.3

Istruzione SQL

- Una volta assegnati i valori ai parametri, l'istruzione può eseguita
- Distinguiamo due tipi di operazioni:
 - Modifiche (insert, update, delete)
 - modificano lo stato del database
 - Interrogazioni (select)
 - non modificano lo stato del database
 - ritornano una sequenza di tuple

Operazione n.3

Istruzione SQL

- Aggiornamenti (insert, delete, update)
 - vengono eseguiti invocando il metodo **executeUpdate()** sull'oggetto **PreparedStatement**
- Esempio 1 (cont.)

```
PreparedStatement statement;  
String insert = "insert into studente(nome, cognome,  
dataNascita, matricola) values (?, ?, ?, ?)";  
statement = connection.prepareStatement(insert);  
statement.setString(1, studente.getNome());  
statement.setString(2, studente.getCognome());  
long secs = studente.getDataNascita().getTime();  
statement.setDate(3, new java.sql.Date(secs));  
statement.setString(4, studente.getMatricola());
```

```
statement.executeUpdate();
```

Operazione n.3

Istruzione SQL

- Interrogazioni (select)
 - vengono eseguite invocando il metodo **executeQuery()**
 - che ritorna il risultato in un oggetto **ResultSet**
- Esempio 2

```
PreparedStatement statement;  
String query = "select * from studente  
               where matricola=?";  
statement = connection.prepareStatement(query);  
statement.setString(1, matricola);  
ResultSet result = statement.executeQuery();
```

Operazione n.4

Gestire il risultato di una query

- Un oggetto della classe **ResultSet** rappresenta la collezione di ennuple restituita da una query SQL (istruzione **SELECT**)
- Per gestire il risultato offre vari metodi:
 - metodo **boolean next()** per scorrere le ennuple (analogo ad un iteratore)
 - metodi **getXXX(String attributo)** per acquisire i valori degli attributi
 - Es.: `int getInt(String attributo);`
 - Es.: `String getString(String attributo);`

Operazione n.4

Gestire il risultato di una query

- Il metodo **next()**
 - Moves the cursor forward one row from its current position. A **ResultSet** cursor is initially positioned before the first row; the first call to the method **next** makes the first row the current row; the second call makes the second row the current row, and so on.
 - When a call to the **next** method returns **false**, the cursor is positioned after the last row. Any invocation of a **ResultSet** method which requires a current row will result in a **SQLException** being thrown.
 - Returns: **true** if the new current row is valid; **false** if there are no more rows

Operazione n.4

Gestire il risultato di una query

```
public List<Studente> findAll() {
    Connection connection = this.dataSource.getConnection();
    List<Studente> studenti = new LinkedList<>();
    try {
        Studente studente;
        PreparedStatement statement;
        String query = "select * from studente";
        statement = connection.prepareStatement(query);
        ResultSet result = statement.executeQuery();
        while (result.next()) {
            studente = new Studente();
            studente.setMatricola(result.getString("matricola"));
            studente.setNome(result.getString("nome"));
            studente.setCognome(result.getString("cognome"));
            long secs = result.getDate("datanascita").getTime();
            studente.setDataNascita(new java.util.Date(secs));
            studenti.add(studente);
        }
    } catch (SQLException e) {
        throw new PersistenceException(e.getMessage());
    } finally {
        try {
            connection.close();
        } catch (SQLException e) {
            throw new PersistenceException(e.getMessage());
        }
    }
    return studenti;
}
```

Operazione n.4

Gestire il risultato di una query

```
List<Studente> studenti = new LinkedList<>;  
...  
  
Studente studente;  
PreparedStatement statement;  
String query = "select * from studente";  
statement = connection.prepareStatement(query);  
ResultSet result = statement.executeQuery();  
while (result.next()) {  
    studente = new Studente();  
    studente.setMatricola(result.getString("matricola"));  
    studente.setNome(result.getString("nome"));  
    studente.setCognome(result.getString("cognome"));  
    long secs = result.getDate("datanascita").getTime();  
    studente.setDataNascita(new java.util.Date(secs));  
    studenti.add(studente);  
}
```

Operazione n.5

Gestire le eccezioni

- Tutti i metodi delle classi dell'API JDBC viste fino ad ora lanciano una eccezione **SQLException**
- Connessione, statement, resultSet devono essere sempre "chiusi" dopo essere stati usati
 - l'operazione di chiusura corrisponde al rilascio di risorse
 - Effettuiamo queste operazioni nella clausola **finally**
 - abbiamo così la garanzia che vengono comunque effettuate

Aspetti Metodologici

- Gestione delle associazioni
 - Problemi (discussi nella lezione su ORM)
 - Lazy Load (qui vediamo come si realizza tecnicamente)

Rappresentazioni Grafiche

Classe

Studente
-matricola: String -nome: String -cognome: String -dataNascita: Date

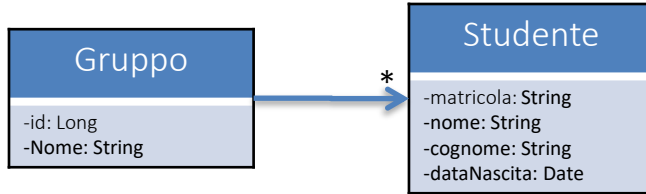
Schema Tabella

studente		
matricola	varchar(255)	nullable=false PK
nome	varchar(255)	nullable=false
cognome	varchar(255)	nullable=false
datanascita	date	nullable=true

Istanza Tabella

STUDENTE		
matricola	nome	...
121212	Robert	...
232323	Julie	...
343434	John	...
...	...	...

Esempio di riferimento



gruppo		
id	bigint	nullable=false PK
nome	varchar(255)	nullable=false

studente		
matricola	character(8)	nullable=false PK
nome	varchar(255)	nullable=false
cognme	varchar(255)	nullable=false
data_nascita	date	nullable=true
gruppo_id	bigint	FK gruppo.id

Esempio di riferimento

gruppo		
id	bigint	nullable=false PK
nome	varchar(255)	nullable=false

studente		
matricola	character(8)	nullable=false PK
nome	varchar(255)	nullable=false
cognome	varchar(255)	nullable=false
data_nascita	date	nullable=true
gruppo_id	bigint	FK gruppo.id

gruppo	
ID	NOME
1	Le Mele
2	Microgeeks
...	...

studente				
matricola	nome	cognome	data_nascita	gruppo_id
00000001	Roberto	Bianchi	20/05/1997	1
00000002	Gilia	Rossi	12/02/1997	1
...	...	...	...	

Esempio di riferimento

```
public class Gruppo {  
    private Long id;  
    private String nome;  
    private Set<Studente> studenti;  
  
    public Gruppo(){  
        this.studenti = new HashSet<>();  
    }  
  
    // getter e setter  
  
    public void addStudente(Studente studente){  
        this.getStudenti().add(studente);  
    }  
  
    public void removeStudente(Studente studente){  
        this.getStudenti().remove(studente);  
    }  
}
```

Esempio di riferimento

```
public class Studente {  
    private String matricola;  
    private String nome;  
    private String cognome;  
    private Date dataNascita;  
  
    public Studente(){}  
  
    // getter e setter  
    // hashCode e equals  
  
}
```

Ricostruzione degli oggetti dal db

- Consideriamo la ricostruzione di un oggetto **Gruppo**, ad esempio per definire il metodo

Gruppo findById(Long id)

- nella classe **Gruppo**:
 - una collezione di riferimenti a oggetti **Studiante**;
- nel database:
 - tabella **gruppo**
 - tabella **studiante** (foreign key con **gruppo**)
- Per ricostruire un oggetto **Gruppo** "completo" devo ricostruire la collezione di oggetti **Studiante** (e anche questi oggetti devono essere "completi"?!)

Ricostruzione degli oggetti dal db

- Come ricostruiamo l'oggetto **Gruppo** "completo"?
- Se voglio ricostruire la collezione di riferimenti agli studenti
 - eseguo un join tra le tabelle **gruppo** e **studente**
 - itero sul **ResultSet** e costruisco i vari oggetti **Gruppo** e **Studente** (aggiungendone i riferimenti alla collezione)
- NB: che join usiamo?
 - è possibile che nella tabella **gruppo** ci siano tuple per le quali non ci sono righe nella tabella **studente**?
 - se la risposta è SI, allora devo usare un *outer join*

Esempio

- Scriviamo la classe GruppoDAO
- Concentriamoci sul metodo:
 - **Gruppo findById(Long id)**

Esempio con Join

```
public Gruppo findById(Long id) {  
    Connection connection = this.dataSource.getConnection();  
    Gruppo gruppo = null;  
    try {  
        PreparedStatement statement;  
        String query = "SELECT g.id AS g_id, g.nome AS g_nome, "  
                        + "s.matricola AS matricola, s.nome AS nome, "  
                        + "s.cognome AS cognome, "  
                        + "s.datanascita AS datanascita "  
                        + "FROM gruppo g LEFT OUTER JOIN "  
                        + "studente s ON g.id=s.gruppo_id "  
                        + "WHERE g.nome = ?";  
        statement = connection.prepareStatement(query);  
        statement.setLong(1, id);  
        ResultSet result = statement.executeQuery();
```

Esempio con Join (cont.)

```
boolean primaRiga = true;
while (result.next()) {
    if (primaRiga) {
        gruppo = new Gruppo();
        gruppo.setId(result.getLong("g_id"));
        gruppo.setNome(result.getString("g_nome"));
        primaRiga = false;
    }
    if(result.getString("matricola")!=null){
        Studente studente = new Studente();
        studente.setMatricola(result.getString("matricola"));
        studente.setNome(result.getString("nome"));
        studente.setCognome(result.getString("cognome"));
        long secs = result.getDate("datanascita").getTime();
        studente.setDataNascita(new java.util.Date(secs));
        gruppo.addStudente(studente);
    }
}
} catch (SQLException e) { ///...
```

Esempio con Join: note

- Query SQL:

```
SELECT g.id AS g_id, g.nome AS g_nome,  
       s.matricola AS matricola, s.nome AS nome,  
       s.cognome AS cognome,  
       s.datanascita AS datanascita  
FROM gruppo g LEFT OUTER JOIN studente s  
      ON g.id=s.gruppo_id  
WHERE g.nome = ?
```

- Perchè LEFT OUTER JOIN?
 - gruppi senza studenti non verrebbero trovati
- Perchè gli alias nella target list?
 - JDBC fa riferimento ai nomi delle colonne

Problemi soluzione Join

- La soluzione con il join ha un (potenzialmente grosso) problema di prestazioni
- Non è scontato che le operazioni che devo fare sui dati in memoria richiedano i dati degli oggetti collegati (ad esempio mi servono solo i nomi dei gruppi)
- Per ovviare a questo problema, facciamo in modo che gli oggetti della collezione siano caricati solo se necessario, cioè solo se viene invocato un metodo che richiede l'accesso alla collezioni
 - nel nostro esempio il metodo `getStudenti()`

Ricostruzione degli oggetti dal db

- Ma che cosa è un "oggetto completo"?
 - con un modello di dominio complesso questa strategia può portarci a caricare una rete molto vasta di oggetti
(esempio: Professore ← Corso ← Gruppo → Studente → ...)
- Fino a quando devo costruire oggetti "inseguendo i riferimenti"?
 - e quindi calcolando costosi join?

Ricostruzione degli oggetti dal db

- Possiamo dare una risposta a queste domande analizzando i casi d'uso e il modello del dominio
- In particolare l'inseguimento dei riferimenti tra oggetti è guidato dalle operazioni offerte dal sistema
- Nel progettare le classi, i riferimenti vanno messi considerando la direzione in cui le corrispondenti associazioni sono navigate dalle operazioni del sistema
 - questo aspetto non ha rilevanza nella progettazione del database, perché i riferimenti sono implementati tramite valori e l'operatore di join non è direzionale

Ricostruzione degli oggetti dal db

- E quando non ha senso costruire un oggetto completo, come mi comporto?
 - (i metodi che richiedono l'inseguimento di un riferimento, se invocati devono ritornare l'oggetto corretto!)
- In questo caso ricorriamo ad una tecnica, detta *Caricamento Pigro*, implementata tramite l'applicazione del *Pattern Proxy*

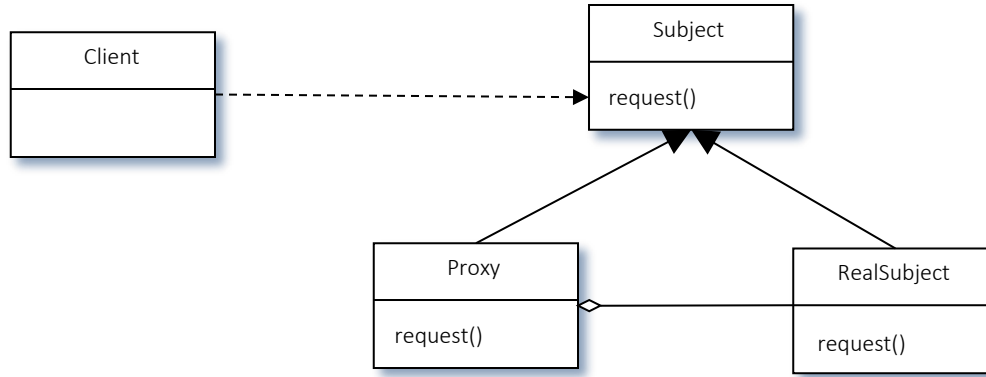
Lazy Load (caricamento pigro)

- Consideriamo la classe **Gruppo**
- Se non è ragionevole pensare che per caricare i dati di un gruppo si debba ricostruire l'intera rete di oggetti collegati dai riferimenti nelle collezioni, possiamo pensare che la collezione di studenti associati al gruppo possa essere caricata solo quando richiesta:
 - quando invochiamo il metodo **getStudenti ()** su un oggetto **Gruppo**
- Questa strategia si chiama *Lazy Load* (caricamento pigro)

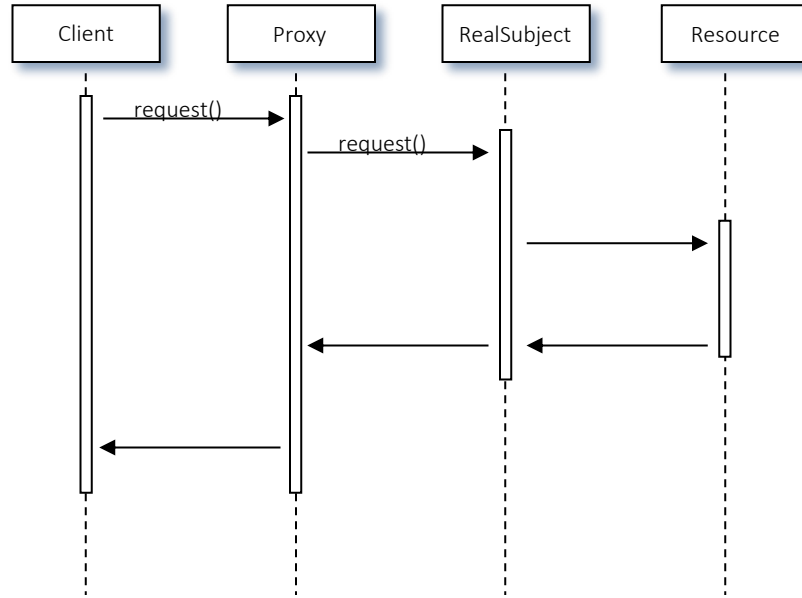
Lazy Load (caricamento pigro)

- In pratica quello che vogliamo fare è caricare i dati solo quando questi sono effettivamente richiesti
- Vediamo come dovrebbe agire il metodo **getStudenti ()** della classe **Gruppo** seguendo questa strategia
 - quando invocato il metodo **getStudenti ()** esegue la query per costruire gli oggetti **Studente** associati al gruppo, aggiunge i riferimenti a questi oggetti nella collezione dell'oggetto **Gruppo**, e restituisce la collezione
- E' importante osservare che se il metodo non viene mai invocato, il costo di caricare i dati relativi agli ordini non viene mai pagato
- Non possiamo però implementare questa strategia nella classe **Gruppo**: metteremmo codice di gestione della persistenza in una classe del dominio
- Per implementare bene la strategia Lazy Load senza ridurre la coesione nella classe del modello è utile fare riferimento al pattern Proxy

II pattern proxy: structure



Pattern Proxy: Participants and Responsibilities



Lazy Load e classi proxy

- I DAO non restituiscono un oggetto “completo” ma un *proxy*
 - Il *proxy* ha il compito di gestire il caricamento degli oggetti collegati
 - Il *proxy* deve contenere tutte le informazioni necessarie per effettuare il caricamento “lazy”
 - Gli oggetti collegati vengono caricati solo su richiesta
 - Il *proxy* nasconde l'accesso al database

Lazy Load: dettagli

- Quando chiediamo ad un DAO di recuperare un oggetto (o una collezione di oggetti), questi esegue una query sul DB e, con i dati recuperati, istanzia e restituisce l'oggetto richiesto
- L'oggetto restituito non è però del tipo richiesto, ma è un proxy, il cui tipo **estende** il quello richiesto e **riscrive** i metodi che devono implementare le operazioni onerose
 - nell'esempio, l'operazione onerosa consiste nel metodo **getStudenti()**

Lazy Load

```
public class GruppoDaoJDBC {  
    // ...  
    public Gruppo findByPrimaryKey(Long id) {  
        Connection connection = this.dataSource.getConnection();  
        Gruppo gruppo = null;  
        try {  
            PreparedStatement statement;  
            String query = "select * from gruppo where id = ?";  
            statement = connection.prepareStatement(query);  
            statement.setLong(1, id);  
            ResultSet result = statement.executeQuery();  
            if (result.next()) {  
                gruppo = new GruppoProxy();  
                gruppo.setId(result.getLong("id"));  
                gruppo.setNome(result.getString("nome"));  
            }  
        } catch (SQLException e) { ... }  
        return gruppo;  
    }  
}
```

Lazy Load: implementazione Proxy

```
public class GruppoProxy extends Gruppo {
    private DataSource dataSource;

    public GruppoProxy() {
        this.dataSource = new DataSource();
    }

    public Set<Studente> getStudenti() {
        Set<Studente> studenti = new HashSet<>();
        Connection connection = this.dataSource.getConnection();
        try {
            PreparedStatement statement;
            String query = "select * from studente where gruppo_id = ?";
            statement = connection.prepareStatement(query);
            statement.setLong(1, this.getId());
            ResultSet result = statement.executeQuery();
            while (result.next()) {
                Studente studente = new Studente();
                studente.setMatricola(result.getString("matricola"));
                studente.setNome(result.getString("nome"));
                studente.setCognome(result.getString("cognome"));
                long secs = result.getDate("datanascita").getTime();
                studente.setDataNascita(new java.util.Date(secs));
                studenti.add(studente);
            }
        } catch (SQLException e) { ...
        }
        this.setStudenti(studenti);
        return super.getStudenti();
    }
}
```



JPA

JPA

- Object Relational Mapping
 - modello di corrispondenza tra classi e tabelle
 - metadati (sintassi per definire i dettagli della corrispondenza)
 - linguaggio di interrogazione (simile a SQL, ma con dot-notation)
 - API e framework
- Varie implementazioni
 - OpenJPA
 - TopLink
 - **Hibernate**
 - EclipseLink
 - ...

Entity

- Applicazione OOP: interazione tra oggetti
 - Oggetti: istanze di classi in memoria centrale
- Chiamiamo **entità** (entity) le classi corrispondenti ad oggetti che devono essere memorizzati in maniera persistente in un db
 - nome molto appropriato (cfr BD: modello ER)
 - in linea di massima corrispondono alle classi del modello di dominio (cfr APS)

Entity

- Un oggetto istanza di un'entità può essere
 - aggiunto nel database
 - recuperato dal database
 - aggiornato nel database
 - cancellato dal database
- In pratica le operazioni CRUD
- JPA (in generale un ORM) offre il supporto per manipolare le entità
 - permette di definire il mapping tra classi Java e tabelle relazionali
 - offre librerie per gestire le operazioni CRUD sugli oggetti istanza di un'entità

Object-Relational-Mapping

- Obiettivo: creare una corrispondenza tra classi e tabelle
- Questa corrispondenza offre una "vista" OO agli sviluppatori, consentendogli di usare in maniera trasparente entità (oggetti persistiti) anziché tabelle

Object-Relational-Mapping

- Idea di base:
 - ogni classe corrisponde ad una tabella
 - ogni attributo (variabile di istanza di tipo primitivo o wrapper, String, tipo data/tempo di java.time) corrisponde ad una colonna
 - ogni oggetto corrisponde ad una riga della tabella

Esempio (1/3)

```
package it.uniroma3.siw.model;
```

```
import javax.persistence.Entity;  
import javax.persistence.GeneratedValue;  
import javax.persistence.GenerationType;  
import javax.persistence.Id;  
import javax.persistence.NamedQuery;  
import javax.persistence.Column;
```

```
@Entity
```

```
@NamedQuery(name = "findAllProducts", query = "SELECT p FROM Product p")
```

```
public class Product {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.AUTO)
```

```
    private Long id;
```

```
    @Column(nullable = false)
```

```
    private String name;
```

```
    private Float price;
```

```
    @Column(length = 2000)
```

```
    private String description;
```

```
    @Column(nullable = false)
```

```
    private String code;
```

Esempio (2/3)

```
public Product() {  
}
```

```
public Product(String name, Float price, String description, String code) {  
    this.name = name;  
    this.price = price;  
    this.description = description;  
    this.code = code;  
}
```

```
public String getDescription() {  
    return description;  
}
```

```
public void setDescription(String description) {  
    this.description = description;  
}
```

```
public String getCode() {  
    return code;  
}
```

```
public void setCode(String code) {  
    this.code = code;  
}
```

```
public void setId(Long id) {  
    this.id = id;  
}
```

Esempio (3/3)

```
public Long getId() {
    return id;
}

public String getName() {
    return this.name;
}

public void setName(String name) {
    this.name = name;
}

public Float getPrice() {
    return price;
}

public void setPrice(Float price) {
    this.price = price;
}

public boolean equals(Object obj) {
    Product product = (Product)obj;
    return this.getCode().equals(product.getCode());
}

public int hashCode() {
    return this.code.hashCode();
}

public String toString() {
    final StringBuilder sb = new StringBuilder();
    sb.append("Product");
    sb.append("{id=").append(id);
    sb.append(", name=").append(name);
    sb.append(", price=").append(price);
    sb.append(", description=").append(description);
    sb.append(", code=").append(code);
    sb.append("}\n");
    return sb.toString();
}
}
```

Esempio (dettagli)

```
package it.uniroma3.siw.model;  
+import javax.persistence.Entity;[...]
```

In rosso le annotazioni JPA per
il mapping object relational

```
@Entity
```

```
@NamedQuery(name = "findAllProducts", query = "SELECT p FROM Product p")
```

```
public class Product {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.AUTO)
```

```
    private Long id;
```

```
    private String name;
```

```
    private Float price;
```

```
    @Column(length = 2000)
```

```
    private String description;
```

```
    @Column(nullable = false)
```

```
    private String code;
```

Esempio

Classe

<<Entity>> Product	
- id:	Long
- name:	String
- price:	Float
- description:	String
- code:	String

mapping
⇐ = ⇒

Tabella

product		
id	bigint	nullable=false PK
name	varchar(255)	nullable=false
price	double	nullable=true
description	varchar(2000)	nullable=true
code	varchar(255)	nullable=false

Rappresentazione testuale in forma compatta:
Product(id, name, price, description, code)

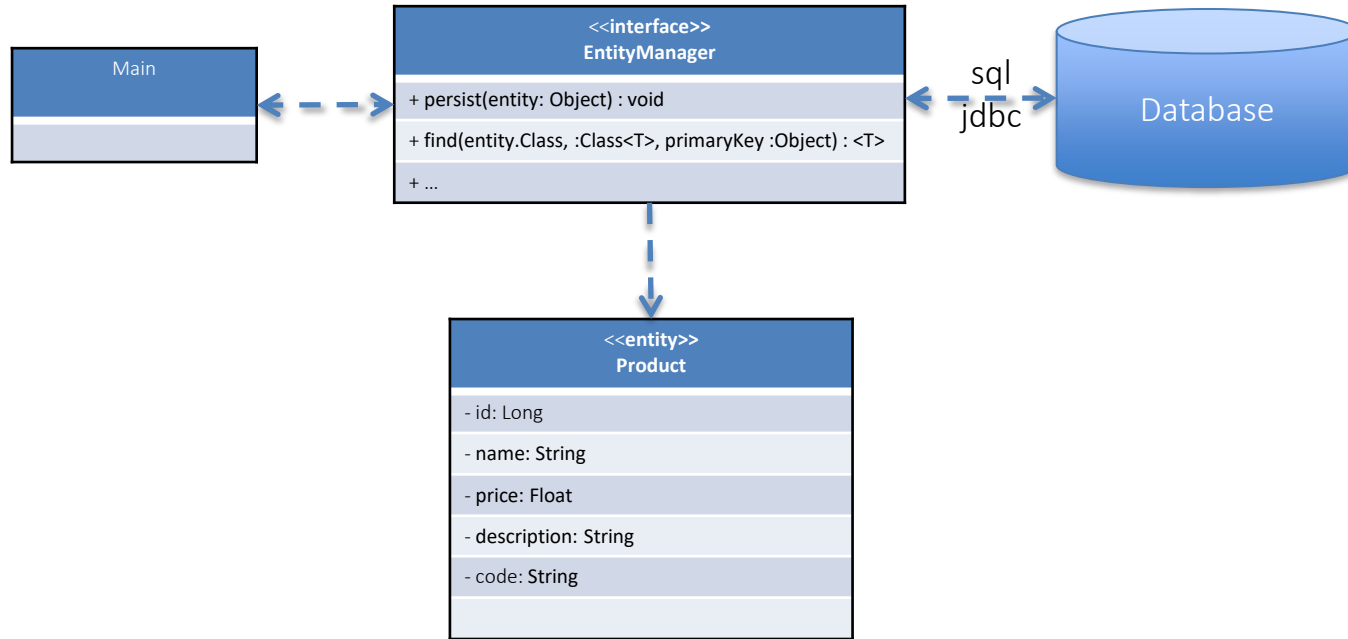
Interrogare le entità

- JPA offre anche uno strumento per interrogare le entità nel db: Java Persistence Query Language (JPQL)
- Una istruzione JPQL può essere creata ed eseguita dinamicamente, oppure definita staticamente
- Nell'esempio abbiamo una query definita staticamente
`@NamedQuery(name = "findAllProducts", query = "SELECT p FROM Product p")`
 - queste query prendono il nome di NamedQuery
 - vanno definite nella classe dell'entità a cui fanno riferimento

EntityManager: introduzione

- Le operazioni di persistenza delle entità sono affidate ad un *gestore delle entità*, una classe istanza di **EntityManager**
- Il ruolo dell'EntityManager è quello di gestire le operazioni di persistenza delle entità: interagire con il db per effettuare le operazioni CRUD sulle entità

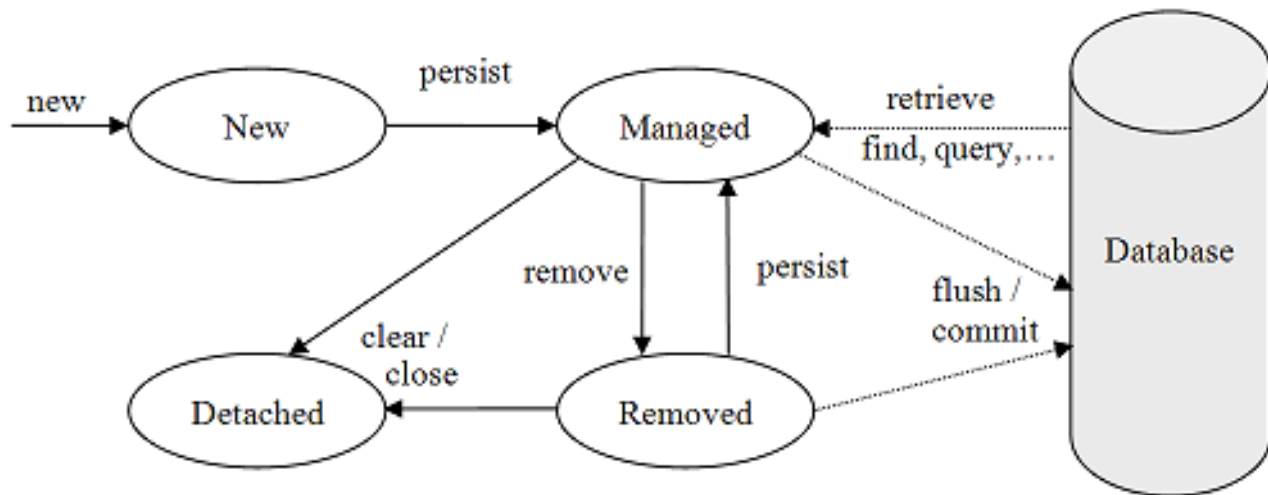
EntityManager: introduzione



EntityManager: introduzione

- Le entità sono semplici classi Java
- Quando un'istanza di una entità è sotto la gestione dell'EntityManager diciamo che si trova nello stato **managed** (o attached, usando l'iniziale terminologia Hibernate)
- Quando un oggetto è gestito dall'EntityManager ha una identità persistente e il suo stato in memoria centrale è sincronizzato con il database
- Le istanze delle entità hanno quindi un "ciclo di vita"

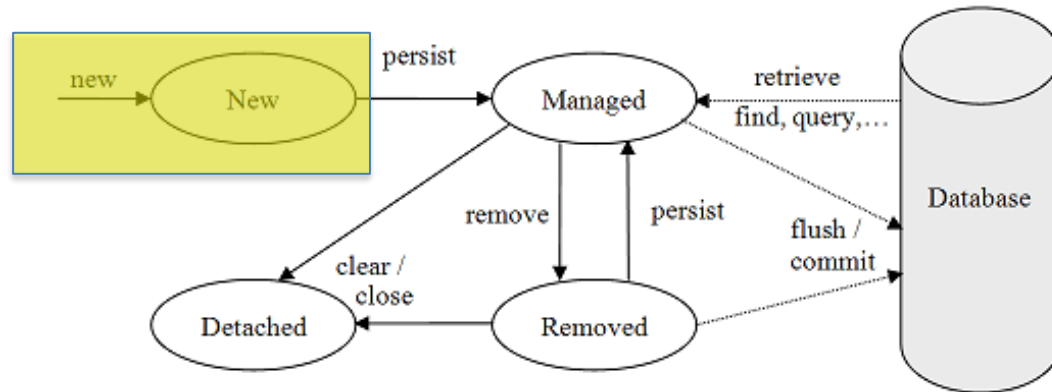
Ciclo di vita delle entità



- Ogni ellisse rappresenta uno stato
- Ogni freccia rappresenta un evento che determina un cambiamento di stato

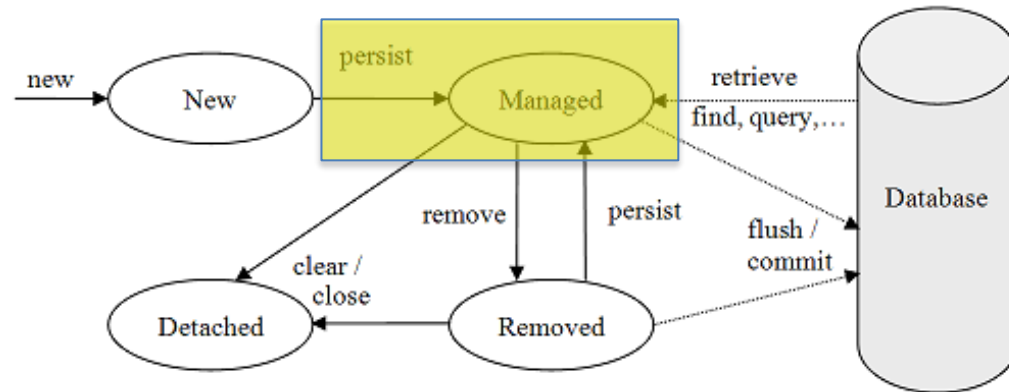
Ciclo di vita delle entità

- Un oggetto appena creato con l'operatore Java new si trova nello stato "**New**":
 - si trova solo in memoria centrale



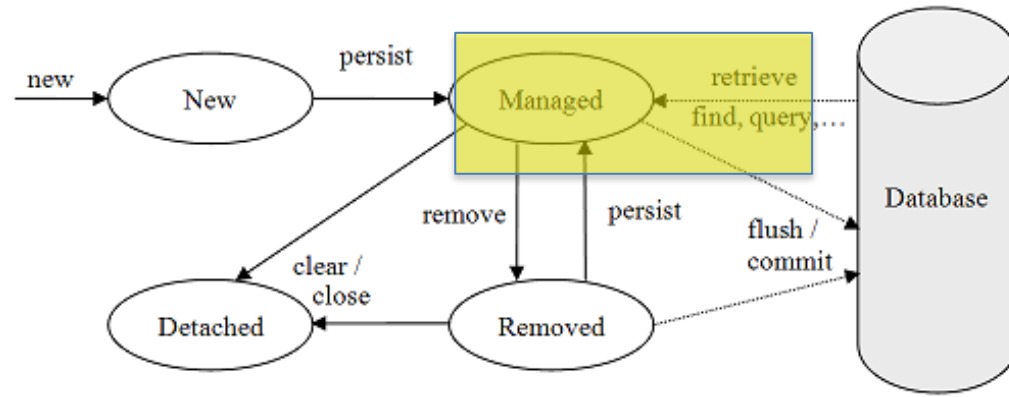
Ciclo di vita delle entità

- Se su un oggetto viene persistito, passa nello stato **Managed**:
 - si trova in memoria centrale
 - una sua copia è memorizzata anche nel db (oggetto in memoria e tupla del db sono sincronizzate)



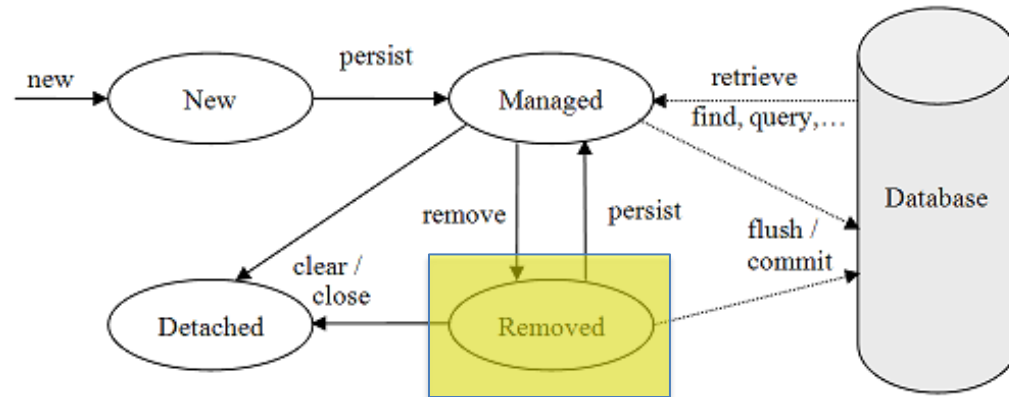
Ciclo di vita delle entità

- Un oggetto creato recuperandone la copia persistita nel db si trova nello stato Managed:
 - si trova in memoria centrale
 - una sua copia è memorizzata anche nel db (oggetto in memoria e tupla del db sono sincronizzate)



Ciclo di vita delle entità

- Un oggetto rimosso dal db si trova nello stato Removed:
 - si trova in memoria centrale, la sua copia nel db non esiste più



EntityManager: introduzione

- Attraverso un esempio scritto **a puro scopo didattico** (che fa uso di una classe, ProductMain, con un unico metodo main()), vediamo:
 - come si crea e si configura l'EntityManager
 - come si affida all'EntityManager la responsabilità di gestire le operazioni CRUD di una entità
 - come si rende persistente una entità

```
package it.uniroma3.siw.main;
```

Esempio

```
import javax.persistence.EntityManager;
```

```
import javax.persistence.EntityManagerFactory;
```

```
import javax.persistence.EntityTransaction;
```

```
import javax.persistence.Persistence;
```

```
import it.uniroma3.siw.model.Product;
```

```
public class ProductMain {
```

```
    public static void main(String[] args) {
```

```
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("products-unit");
```

```
        EntityManager em = emf.createEntityManager();
```



Esempio (cont.)

```
Product product = new Product();  
product.setName("KRIDDIG");  
product.setPrice(3.5F);  
product.setDescription("A wonderful bla bla");  
product.setCode("pt000154");
```

```
EntityTransaction tx = em.getTransaction();  
tx.begin();  
em.persist(product);  
tx.commit();
```

```
em.close();  
emf.close();
```

```
}
```

```
}
```

Persistence Unit

- L'EntityManager è creato per mezzo della factory EntityManagerFactory
- Per ottenere la factory dobbiamo specificare dettagli di configurazione (ad es. come ottenere la connessione al database)
- Questi dettagli si specificano nell'elemento **persistence-unit** di un file di configurazione (scritto in formato XML): **persistence.xml**
- Vedremo in dettaglio come è fatto e come usare questo file tra breve

Transazione

- Ogni operazione che modifica lo stato del db deve essere eseguita in una transazione
 - insieme di operazioni che devono essere effettuate in modo atomico, come se fossero una sola operazione
- In JPA, una transazione:
 - viene creata dall'EntityManager
 - le istruzioni comprese tra `begin()` e `commit()` vengono eseguite in modo atomico sul dbms

persistence.xml

- E' un file XML (una sintassi standard)
- deve essere nella directory del progetto
src/META-INF
- Vediamo un esempio di un file persistence.xml
- La sintassi XML è molto pesante, anche se intuitiva (si basa su marcatori)
 - ma ci soffermeremo sugli elementi essenziali, che sono pochi e semplici

persistence.xml

- Il file persistence.xml si compone di una o più persistence-unit
- Ogni persistence-unit inizia con il marcatore
 <persistence-unit name="*nome della unit*">
e termina con il marcatore
 </persistence>
- Il nome della unità è quello che specifichiamo nella creazione dell'EntityManagerFactory

persistence.xml (esempio)

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.0"
  xmlns="http://java.sun.com/xml/ns/persistence" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence_2_0.xsd">
  <persistence-unit name="products-unit">
    <properties>
      <property name="javax.persistence.jdbc.driver" value="org.postgresql.Driver" />
      <property name="javax.persistence.jdbc.user" value="postgres" />
      <property name="javax.persistence.jdbc.password" value="postgres" />
      <property name="javax.persistence.jdbc.url" value="jdbc:postgresql://localhost/products" />
      <property name="hibernate.show_sql" value="true" />
      <property name="hibernate.format_sql" value="true" />
      <property name="hibernate.dialect" value="org.hibernate.dialect.PostgreSQLDialect" />
      <property name="hibernate.hbm2ddl.auto" value="create" />
    </properties>
  </persistence-unit>
</persistence>
```



persistence.xml

- Ci permette di specificare le proprietà per la connessione al database:

- driver del dbms

- ```
<property name="javax.persistence.jdbc.driver" value="org.postgresql.Driver" />
```

- username e password per accedere al dbms  
(devono essere quelli del vostro dbms!)

- ```
<property name="javax.persistence.jdbc.user" value="postgres" />
```

- ```
<property name="javax.persistence.jdbc.password" value="postgres" />
```

- url del database (host, porta, nome del database)

- ```
<property name="javax.persistence.jdbc.url" value="jdbc:postgresql://localhost/products" />
```

Esecuzione NamedQuery

- Il test ci offre anche un esempio di come l'EntityManager gestisce le interrogazione al db (con l'esecuzione di una NamedEntity)

```
List<Product> products=  
    em.createNamedQuery("findAllProducts",  
        Prodcut.class).getResultList();
```

Operazioni dell'Entity Manager

- Vediamo i metodi principali dell'EM
 - `persist(Object entity)`
 - `remove(Object entity)`
 - `merge(Object entity)`
 - `find(Class<T> entityClass, Object primaryKey)`
- Per dettagli su tutti gli altri metodi si consulti la documentazione Javadoc

EM: persist(Object entity)

- public void persist(Object entity);
 - Transitions new instances to managed. On the next commit (or flush), the newly persisted instances will be inserted into the datastore.
 - For a given entity A, the persist method behaves as follows:
 - If A is a new entity, it becomes managed.
 - If A is an existing managed entity, it is ignored. However, the persist operation cascades.
 - If A is a removed entity, it becomes managed.
 - If A is a detached entity, an IllegalArgumentException is thrown
 - The persist operation recurses on all relation fields of A whose [cascades](#) include CascadeType.PERSIST.
 - This action can only be used in the context of an active transaction.

EM: persist(Object entity)

- Abbiamo già visto che un'operazione persist deve essere effettuata all'interno di una "transazione": operazione atomica (tutto o niente) effettuata sul db
- Internamente ad una transazione possono essere effettuate più operazioni. La transazione garantisce che le sue operazioni vengono effettuate **tutte o nessuna**
- Questo comportamento è fondamentale per garantire la consistenza dei dati nel database, soprattutto in un ambiente concorrente (in cui cioè possono esserci più client che eseguono transazioni)

EM: persist(Object entity)

- Quando una entità è "managed", ogni modifica ai suoi dati (prima del commit) si riflette nel database
- Il valore del nome del prodotto nel db viene aggiornato automaticamente al valore "ARSTANG"

```
Product product = new Product();  
...  
product.setName("POLLUNG");
```

```
EntityTransaction tx = em.getTransaction();
```

```
tx.begin();  
em.persist(product);  
product.setName("ARSTANG")  
tx.commit();
```

EM: persist(Object entity)

- Attenzione però che dobbiamo gestire noi le entità collegate
Esempio:

```
Customer customer = new Customer(...);  
Address address = new Address(...);
```

```
customer.setAddress(address);
```

```
EntityTransaction tx = em.getTransaction();
```

- Senza effettuare la persist dell'istanza address, questa non viene salvata nel db
- Più avanti (vedi "Eventi in Cascata") vedremo come si può semplificare la persistenza delle associazioni

```
tx.begin();  
em.persist(customer);  
em.persist(address);  
tx.commit();
```

Persist con @Id

@GeneratedValue(strategy=GenerationType.AUTO

- **@Id**
specifica che l'attributo è un'identificatore
- **@GeneratedValue**
specifica in che modo deve essere generato il valore della chiave primaria
 - **strategy=GenerationType.AUTO** indica che il valore dell'id deve essere generato automaticamente (un valore incrementale assegnato dal dbms)
 - Il valore dell'id viene creato (automaticamente, dal dbms) la prima volta che l'oggetto viene reso persistente nel database (vedi il test)

EM: remove(Object entity)

- public void remove(Object entity);
 - Transitions managed instances to removed. The instances will be deleted from the datastore on the next commit (or flush). Accessing a removed entity has undefined results.
 - For a given entity A, the remove method behaves as follows:
 - If A is a new entity, it is ignored. However, the remove operation cascades as defined below.
 - If A is an existing managed entity, it becomes removed.
 - If A is a removed entity, it is ignored.
 - If A is a detached entity, an IllegalArgumentException is thrown.
 - The remove operation recurses on all relation fields of A whose [cascades](#) include CascadeType.REMOVE.
 - This action can only be used in the context of an active transaction.

EM: merge(Object entity)

- public Object merge(Object entity);
 - The JPA merge API re-attaches detached entities. It "merges" the state of the given entity into the current persistence context;
 - that is, it updates the data in the datastore with those from the entity
 - This action can only be used in the context of an active transaction.

EM: find(Class<T> entityClass, Object primaryKey)

- T [find](#)(Class<T> entityClass, Object primaryKey)
 - Find by primary key. Search for an entity of the specified class and primary key. If the entity instance is contained in the persistence context, it is returned from there.
 - Parameters: entityClass - entity class primaryKey - primary key
 - Returns: the found entity instance or null if the entity does not exist

EM: clear()

- `public void clear();`
 - All entities managed by the EntityManager become detached.
- Questo metodo è utile per portare tutte le entità nello stato detached

Riassumendo

- I metodi che abbiamo visto corrispondono alle operazioni SQL (e CRUD), come segue:
 - persist → INSERT (Create)
 - find → SELECT (Retrieve)
 - merge → UPDATE (Update)
 - remove → DELETE (Delete)
- I metodi che modificano i dati devono essere invocati nel contesto di una transazione

JPQL: Tipologie di query

- Dynamic Queries: specificate runtime
- Named Queries: statiche e immutabili, sono definite nella entità (vedi prox slide)
- Native Queries: query SQL

Named Queries

- Sono definite in maniera statica e immutabile nella entità a cui fanno riferimento
 - Prima della definizione della classe
 - Tramite l'annotazione
`@NamedQuery("nome", query="jpql query")`
- Possono essere sia query di interrogazione che query di aggiornamento (update o delete)
- Possono essere parametriche

Named Queries: esempio

```
package it.uniroma3.siw.model;
```

```
import javax.persistence.Entity; [...]
```

```
@Entity
```

```
@NamedQuery(name = "findAllProducts", query = "SELECT p FROM Product p")
```

```
@NamedQuery(name = "deleteAllProducts", query = "DELETE Product p")
```

```
public class Product {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.AUTO)
```

```
    private Long id;
```

```
    ....
```

JPQL: gestione query

- In JPQL una query deve essere:
 - creata dall'EM
 - eseguita
- Queste operazioni vanno fatte in modo leggermente diverso per le query:
 - SELECT
 - non modificano lo stato del database
 - ritornano un oggetto o una lista di oggetti
 - UPDATE/DELETE
 - modificano lo stato del database
 - ritornano il numero di righe modificate/cancellate

JPQL: Creazione Query SELECT

- La creazione di una **dynamic** query SELECT è effettuata invocando sull'entity manager il metodo
`TypedQuery<T> createQuery(String, Class)`
- La creazione di una **named** query SELECT è effettuata invocando sull'entity manager il metodo
`TypedQuery<T> createNamedQuery(String, Class)`
- La creazione di una **native** query SELECT è effettuata invocando sull'entity manager il metodo
`Query createNativeQuery(String, Class)`
- Per tutte e tre:
 - Il primo parametro è una stringa con la query (jpql/named/SQL)
 - Il secondo parametro è la classe dell'oggetto ritornato dalla query

JPQL: Creazione Query SELECT

- Creazione di una **dynamic query**:

```
TypedQuery<Product> selectQuery =  
    em.createQuery("SELECT p FROM Product p", Product.class);
```

E' una query JPQL

- Creazione di una **named query**:

```
TypedQuery<Product> selectQuery =  
    em.createNamedQuery("findAllProducts", Product.class);
```

E' il nome di una NamedQuery
definita nella entity (vedi slide
precedente)

- Creazione di una **native query**:

```
Query selectQuery =  
    em.createNativeQuery("SELECT * FROM product", Product.class);
```

E' una query SQL

JPQL: Esecuzione Query SELECT

- L'esecuzione di una dynamic/named (native) query SELECT è effettuata mediante un metodo dell'interface TypedQuery<T> (Query)
 - List<T> getResultList()
 - T getSingleResult()
- NB: nel caso di native query, viene ritornato un oggetto List (non un oggetto List<T>): il compilatore (e l'IDE) ci segnalano un warning. Non è un problema, se non vogliamo il warning usiamo l'annotazione `@SuppressWarnings("unchecked")`

JPQL: Esecuzione Query SELECT

- Creazione ed esecuzione di una **dynamic query**:

```
TypedQuery<Product> selectQuery = em.createQuery("SELECT p FROM Product p", Product.class);  
List<Product> selectQueryResults = selectQuery.getResultList();
```

- Creazione ed esecuzione di una **named query**:

```
TypedQuery<Product> selectQuery = em.createNamedQuery("findAllProducts", Product.class);  
List<Product> selectQueryResults = selectQuery.getResultList();
```

- Creazione ed esecuzione di una **native query**:

```
Query selectQuery = em.createNativeQuery("SELECT * FROM product", Product.class);  
@SuppressWarnings("unchecked") // necessario per una inconsistenza delle API  
List<Product> selectQueryResults = selectQuery.getResultList();
```

JPQL: target list query SELECT

- Nella target list possiamo specificare anche un sottoinsieme dei campi, sempre usando una path-expression
- Nel caso in cui abbiamo un solo elemento, la query ritorna una lista del tipo corrispondente

```
TypedQuery<String> queryNames = em.createQuery("SELECT p.name FROM Product p", String.class);  
List<String> names = queryNames.getResultList();
```

```
TypedQuery<Float> queryPrices = em.createQuery("SELECT p.price FROM Product p", Float.class);  
List<Float> prices = queryPrices.getResultList();
```

JPQL: target list query SELECT

- Se abbiamo più elementi, la query ritorna una lista di `Object[]`. Ogni elemento della lista corrisponde ad un array che ha tanti elementi quante le path-expression nella target, e in ogni array mantengono la stessa posizione data dalla target list

```
TypedQuery<Object[]> query = em.createQuery("SELECT p.name, p.price FROM Product p", Object[].class);  
List<Object[]> namesAndPrices = query.getResultList();
```

```
for (Object[] nAp: namesAndPrices) {  
    System.out.println(nAp[0] + " " + nAp[1]);  
}
```

JPQL: Query DELETE/UPDATE

- La creazione e l'esecuzione di una query UPDATE o DELETE vengono fatte in maniera simile
- Tuttavia, l'esecuzione di una query UPDATE/DELETE deve essere effettuata in una transazione (perchè modifica il database)

JPQL: Creazione Query UPDATE/DELETE

- La creazione di una **dynamic** query UPDATE/DELETE è effettuata invocando sull'entity manager il metodo
`Query createQuery(String)`
- La creazione di una **named** query UPDATE/DELETE è effettuata invocando sull'entity manager il metodo
`Query createNamedQuery(String)`
- La creazione di una **native** query UPDATE/DELETE è effettuata invocando sull'entity manager il metodo
`Query createNativeQuery(String)`
- Per tutte e tre:
 - Il parametro è una stringa con la query jpql/named/SQL

JPQL: Creazione Query UPDATE/DELETE

- Creazione di una **dynamic query DELETE**:

Query deleteQuery = *em.createQuery*("DELETE FROM Product p");

E' una query JPQL

- Creazione di una **named query DELETE**:

Query deleteQuery = *em.createNamedQuery*("deleteAllProducts");

E' il nome di una NamedQuery definita nella entity (vedi slide precedente)

- Creazione di una **native query DELETE**:

Query deleteQuery = *em.createNativeQuery*("DELETE FROM product");

E' una query SQL

JPQL: Esecuzione Query UPDATE/DELETE

- L'esecuzione di una dynamic/named/native query UPDATE/DELETE mediante il metodo dell'interface Query
 - `int executeUpdate()`
- Il metodo ritorna il numero di righe che sono state aggiornate/cancellate nel database
- L'esecuzione di una query UPDATE/DELETE deve essere effettuata in una transazione (perchè modifica il database)

JPQL: Esecuzione Query UPDATE/DELETE

Esecuzione di una **dynamic/named/native** query UPDATE/DELETE:

```
tx.begin();
```

```
int deletedRows = deleteQuery.executeUpdate();
```

```
tx.commit();
```

Repository



Repository

- Il pattern Repository viene usato negli approcci Domain Driven Design (DDD)
- I repository sono oggetti che incapsulano la logica necessaria per l'accesso al database
- Favoriscono la manutenibilità del codice disaccoppiando la tecnologia utilizzata per accedere al database dal modello di dominio

Il pattern Repository

- Martin Fowler describe il pattern repository così:
 - *A repository performs the tasks of an **intermediary between the domain model layers and data mapping**, acting in a similar way to a **set of domain objects in memory***
 - *Conceptually, a repository encapsulates a set of objects stored in the database and operations that can be performed on them, providing a way that is closer to the persistence layer*
 - *Repositories, also, support the purpose of separating, clearly and in one direction, the dependency between the work domain and the data allocation or mapping*

Il pattern Repository

- Possiamo pensare un repository come ad una collezione persistente
 - Il programmatore/utilizzatore usa i repository senza che gli sia richiesto di conoscere i dettagli di come internamente viene gestita la persistenza
- Definiamo una classe Repository per (la radice di) ogni aggregato del modello di dominio
- Il repository offre metodi per salvare, recuperare, cancellare oggetti dalla collezione, più altre funzioni di utilità (ad es., conteggio, verifica di esistenza)

Il pattern Repository

- I metodi delle classi repository non devono essere transazionali, la transazione deve essere aperta (e chiusa) dall'oggetto che invoca il metodo del repository
- Infatti, le operazioni offerte dal sistema (che usano i metodi dei repository) possono essere composte da più operazioni fornite da repository diversi: la transazione deve poter essere aperta e chiusa da chi usa i repository

Esempio (didattico) di implementazione

- Vediamo come potremmo implementare il pattern prendendo spunto dalle entità che abbiamo già utilizzato nelle lezioni precedenti
- Da un punto di vista della organizzazione dei package, mettiamo le classi repository in un apposito package (repository)
- Scriviamo la classe ProductRepository, in modo che offra metodi per
 - salvare (o modificare) entità Product nel repository
 - recuperare entità Product dal repository
 - rimuovere entità Product dal repository
 - contare il numero di entità Product presenti nel repository

Esempio

```
package it.uniroma3.siw.repository;
```

```
import java.util.List;
```

```
import javax.persistence.EntityManager;
```

```
import it.uniroma3.siw.model.Product;
```

```
public class ProductRepository {
```

```
    private EntityManager em;
```

```
    public Product save(Product product) {
```

```
        if (product.getId() != null) {
```

```
            em.merge(product);
```

```
        }
```

```
        else {
```

```
            em.persist(product);
```

```
        }
```

```
        return product;
```

```
    }
```

```
    public Product findById(Long id) {
```

```
        return em.find(Product.class, id);
```

```
    }
```

Esempio

```
public List<Product> findAll() {  
    return em.createQuery("select p from Product p", Product.class).getResultList();  
}  
  
public void delete(Product product) {  
    em.remove(product);  
}  
  
public void deleteAll(){  
    this.em.createQuery("delete from Product").executeUpdate();  
}  
  
public long count() {  
    return (Long)this.em.createQuery("select count(id) from Product").getSingleResult();  
}  
  
public boolean existsById(Long id) {  
    return (this.findById(id)!=null);  
}  
  
public void setEntityManager(EntityManager em) {  
    this.em = em;  
}  
}
```

Repository

- Alla struttura di base, che offre operazioni generiche valide per ogni classe del modello, si possono aggiungere operazioni specifiche, (interrogazioni e/o aggiornamenti) basate su criteri che emergono dall'analisi dei requisiti
- Ad esempio:
 - Interrogazioni con selezioni su valori specifici
 - Interrogazioni con aggregazioni
 - etc
- NB: **non è efficiente** (in molti casi potrebbe essere impraticabile) caricare tutti i dati tramite una `findAll()` e poi filtrare i dati in memoria (il volume dei dati potrebbe essere addirittura superiore alla capacità della memoria centrale). Questa operazione la deve fare il DBMS!

Esempio

```
package it.uniroma3.siw.repository;

import [...];

public class ProductRepository {
    ...
    public List<Product> findByName(String name) {...}
    public List<Product> findByNameAndPrice(String name,
                                             float price) {...}
    public List<Product> findByProvider(Provider provider) {...}
}
```

Miglioriamo la soluzione

- Abbiamo tante classi con codice estremamente simile: cambia solo il tipo!
- Possiamo definire una Java interface generica ed una sua implementazione generica
- Ogni repository può essere definito come una classe che estende l'implementazione della interface generica (se necessario, introducendo nuovi metodi)
- Per semplicità, ipotizziamo che le classi del modello abbiano sempre un id (potrebbero implementare un'opportuna interfaccia)

```
public interface SimpleRepository<T> {  
    public EntityManager getEntityManager();  
  
    public void setEntityManager(EntityManager em);  
  
    public T save(T entity);  
  
    public List<T> findAll();  
  
    public T findById(Long id);  
  
    public void delete(T t);  
  
    public void deleteAll();  
  
    public long count();  
  
    public boolean existsById(Long id);  
}
```

SimpleRepository<T>

```
public class SimpleRepositoryImpl<T> implements SimpleRepository<T> {
```

```
    private EntityManager em;  
    private Class<T> domainClass;
```

```
    public SimpleRepositoryImpl(Class<T> domainClass) {  
        this.domainClass = domainClass;  
    }
```

```
    public T save(T entity) {  
        T persistentEntity = null;  
        //TODO (esercizio) // vedi slide 22 e 23  
        return persistentEntity;  
    }
```

```
    public List<T> findAll() {  
        return em.createQuery("select o from " + this.domainClass.getName() + " o", this.domainClass).getResultList();  
    }
```

```
    public T findById(Long id){  
        return em.find(this.domainClass, id);  
    }
```

```
// continua nella pagina successiva
```

SimpleRepositoryImpl<T>

SimpleRepositoryImpl<T>

```
public T findById(Long id){  
    return em.find(this.domainClass, id);  
}
```

```
public void delete(T t){  
    this.em.remove(t);  
}
```

```
public void deleteAll(){  
    this.em.createQuery("DELETE FROM "+this.domainClass.getName()).executeUpdate();  
}
```

```
public long count() {  
    return (long)this.em.createQuery("SELECT COUNT(id) FROM "+this.domainClass.getName()).getSingleResult();  
}
```

```
public boolean existsById(Long id) {  
    return (this.findById(id)!=null);  
}
```

```
}
```

Semplice implementazione del metodo save():
assume che nella classe dell'entità
esista un metodo che si chiama getId()

```
public T save(T entity) {  
    Method getId = null;  
    T persistentEntity = null;  
    try {  
        getId = this.domainClass.getMethod("getId");  
    } catch (NoSuchMethodException | SecurityException e) {  
        e.printStackTrace();  
    }  
    try {  
        if (getId.invoke(entity) == null) { // (entity.getId() == null)  
            this.em.persist(entity);  
            persistentEntity = entity;  
        } else {  
            persistentEntity = em.merge(entity);  
        }  
    } catch (IllegalAccessException | IllegalArgumentException | InvocationTargetException e) {  
        e.printStackTrace();  
    }  
    return persistentEntity;  
}
```

```

private String getNameOfIdentifier() {
    String fieldName = null;
    for (Field f : domainClass.getDeclaredFields()) {
        fieldName = f.getName();
        if (f.isAnnotationPresent(Id.class))
            return "get" + Character.toUpperCase(fieldName.charAt(0)) + fieldName.substring(1);
    }
    return null;
}

```

```

public T save(T entity) {
    Method getId = null;
    T persistentEntity = null;
    try {
        getId = this.domainClass.getMethod(getNameOfIdentifier());
    } catch (NoSuchMethodException | SecurityException e) {
        e.printStackTrace();
    }
    try {
        if (getId.invoke(entity) == null) { // (entity.getId() == null)
            this.em.persist(entity);
            persistentEntity = entity;
        } else {
            persistentEntity = em.merge(entity);
        }
    } catch (IllegalAccessException | IllegalArgumentException | InvocationTargetException e) {
        e.printStackTrace();
    }
    return persistentEntity;
}

```

Implementazione più avanzata del metodo save():
 usa la riflessione per cercare nella classe dell'entità
 la variabile annotata @Id

CustomerRepository

```
public class CustomerRepository extends SimpleRepositoryImpl<Customer> {  
  
    private static final String SELECT_BY_NAME = "select c from Customer c where name = :name";  
  
    public CustomerRepository() {  
        super(Customer.class);  
    }  
  
    public List<Customer> findByName(String name) {  
        TypedQuery<Customer> query = null;  
        query = this.getEntityManager().createQuery(SELECT_BY_NAME , Customer.class);  
        query.setParameter("name", name);  
        return query.getResultList();  
    }  
}
```



CrudRepository